UNIVASSOURAS

CAMPUS UNIVERSITÁRIO
DE SAQUAREMA

Curso de Graduação em Engenharia de Software

# Estrutura de Dados

## Listas e Pilhas

## Tópicos

### Lista Linear

- Uma estrutura de dado armazena dados na memória do computador a fim de permitir o acesso eficiente dos mesmos.
- A maioria das estruturas de dados consideram a memória primária (a chamada RAM) como pilhas, filas, árvores binárias de busca, árvores AVL e árvores rubro-negras.
- Uma estrutura de dado bem projetada permite a manipulação eficiente, em tempo e em espaço, dos dados armazenados através de operações específicas. Um conceito relacionado com a estrutura de dado é o tipo abstrato de dados

## Tópicos

### Lista Linear

- Uma lista linear é um conjunto de n elementos (de informações)

$$x_1, x_2, \ldots, x_n,$$

- cuja propriedade estrutural envolve as posições relativas de seus elementos. Supondo n > 0, temos
    - x1 é o primeiro elemento
    - para $1 < k < n$, $x_k$ é precedido por $x_{k-1}$ e seguido por $x_{k+1}$
    - $x_n$ é o último elemento.

## Tópicos

### Lista Linear

- Algumas operações que podemos querer realizar sobre listas lineares:
  - Ter acesso a $x_k$ , *k* qualquer, a fim de consultar ou alterar o conteúdo de seus campos
  - Inserir um elemento novo antes ou depois de $x_k$
  - Remover $x_k$
  - Colocar todos os elementos da lista em ordem.
  - Combinar 2 ou mais listas lineares em uma só
  - Quebrar uma lista linear em duas ou mais
  - Copiar uma lista linear em um outro espaço
  - Determinar o número de elementos de uma lista linear

## Tópicos

### Lista Linear

- Trataremos neste curso as operações abaixo, para k = 1 e k = n, casos considerados importantes e as listas lineares recebem nomes como pilha ou fila conforme a maneira essas operações são realizadas.

  - Ter acesso a $x_k$ , *k* qualquer, a fim de consultar ou alterar o conteúdo de seus campos
  - Inserir um elemento novo antes ou depois de $x_k$
  - Remover $x_k$

## Tópicos

### Lista Linear

- A maneira mais simples de manter uma lista linear é usando um vetor (alocação sequencial).
    - Como a alocação sequencial da maioria das linguagens geralmente é realizada com a reserva prévia de memória para cada estrutura, a inserção e a remoção de nós não ocorrem de fato.
    - Normalmente é feita uma simulação das operações de inserção e remoção. É comum delimitar os limites da estrutura utilizando variáveis.

## Tópicos

### Pilha (Stack)

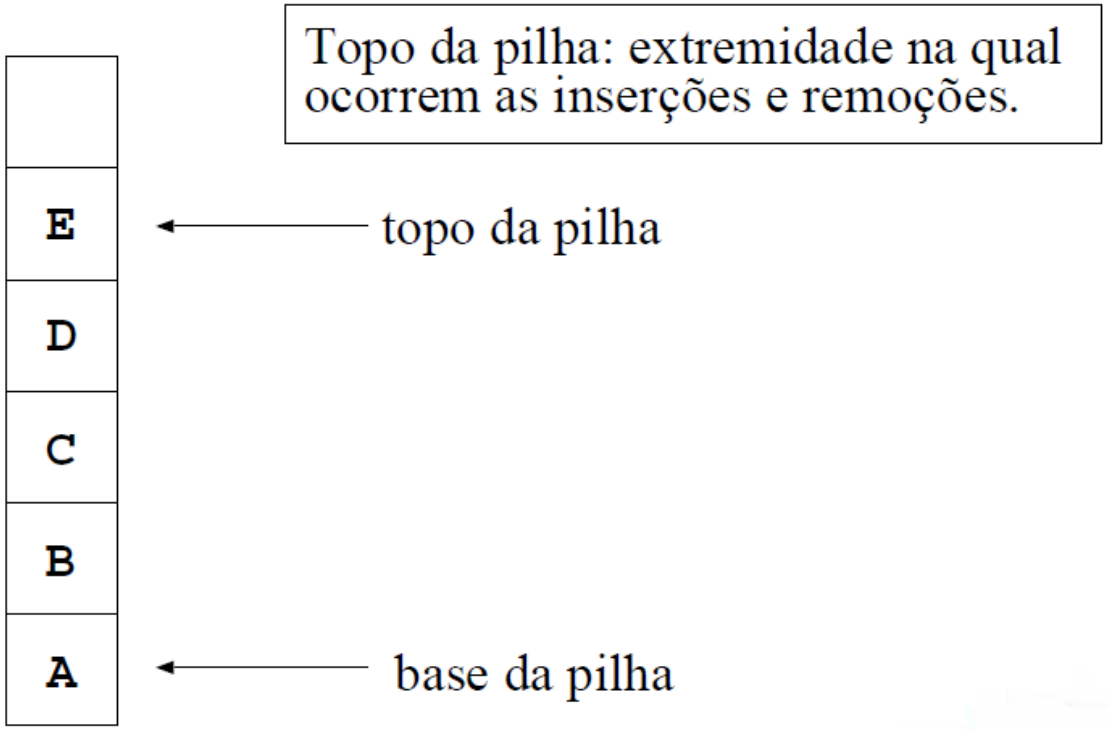
- Listas lineares em que inserções, remoções e acessos a elementos ocorrem no primeiro ou no último elemento são muito frequentemente encontradas. Tais listas recebem nomes especiais como pilha e fila.

- Na Pilha todas as inserções e remoções são feitas numa mesma extremidade. Esta extremidade se denomina topo.

- As operações definidas para uma pilha incluem:
    - Verificar se a pilha está vazia
    - Inserir um elemento na pilha (empilhar ou “push”), no lado do topo.
    - Remover um elemento da pilha (desempilhar ou “pop”), do lado do topo.

## Tópicos

- Pilha (Stack)



## Tópicos

### Pilha (Stack)

- Situações extremas:
  - pilha vazia;
  - pilha cheia.

- Operações inválidas:
  - inserção em pilha cheia (overflow)
  - remoção em pilha vazia (underflow)

- Forma de operação:
  - último a entrar, primeiro a sair (LIFO)

## Tópicos

### Pilha (Stack)

- Como o último elemento que entrou na pilha será o primeiro a sair da pilha, a pilha é conhecida como uma estrutura do tipo LIFO (“Last In First Out”). Exemplos:
    - Na vida real: pilhas de pratos numa cafeteria (acréscimos e retiradas de pratos sempre feitos num mesmo lado da pilha - lado de cima)
    - Na execução de uma programa: uma pilha pode ser usada na chamada de procedimentos, para armazenar o endereço de retorno (e os parâmetros reais). A medida que procedimentos chamam outros procedimentos, mais e mais endereços de retorno devem ser empilhados. Estes são desempilhados à medida que os procedimentos chegam ao seu fim.

## Tópicos

### Pilha (Stack)

- Como operações, damos um exemplo sem maiores detalhes.

- **Operações:**
  - ✓ Push (insere)
  - ✓ Pop (remove)
  - ✓ Top (topo)
  - ✓ Clear (apaga todos os elementos)
  - ✓ Empty (verifica se está vazia)

- **Condições pre e pos:** estando **pre** verificada, **pos** é verificada após a operação.

## Tópicos

### Pilha (Stack)

- **Operações:**
  - ✓ Push (insere)

    pre verifica se não está cheia

    pos verifica se ocorreu a inserção

  - ✓ Pop (remove)
  - ✓ Top (topo)
  - ✓ Clear (apaga todos os elementos)
  - ✓ Empty (verifica se está vazia)

- **Condições pre e pos:** estando **pre** verificada, **pos** é verificada após a operação.

## Tópicos

### Pilha (Stack)

- **Operações:**
  - ✓ Push (insere)
  - ✓ Pop (remove)

    pre verifica se não está vazia

    pos verifica se ocorreu a remoção

  - ✓ Top (topo)
  - ✓ Clear (apaga todos os elementos)
  - ✓ Empty (verifica se está vazia)

- **Condições pre e pos:** estando **pre** verificada, **pos** é verificada após a operação.

## Tópicos

### Pilha (Stack)

- **Operações:**
  - ✓ Push (insere)
  - ✓ Pop (remove)
  - ✓ Top (topo)

    pre verifica se não está vazia

    pos retorna o índice do elemento na pilha

  - ✓ Clear (apaga todos os elementos)
  - ✓ Empty (verifica se está vazia)

- **Condições pre e pos:** estando **pre** verificada, **pos** é verificada após a operação.

## Tópicos

### Pilha (Stack)

- **Operações:**
  - ✓ Push (insere)
  - ✓ Pop (remove)
  - ✓ Top (topo)
  - ✓ Clear (apaga todos os elementos)

    pos retorna a pilha vazia

  - ✓ Empty (verifica se está vazia)

- **Condições pre e pos:** estando **pre** verificada, **pos** é verificada após a operação.

## Tópicos

### Pilha (Stack)

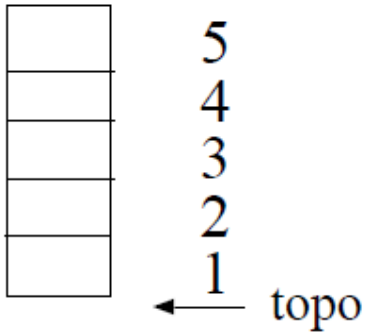
- **Operações:**
  - ✓ Push (insere)
  - ✓ Pop (remove)
  - ✓ Top (topo)
  - ✓ Clear (apaga todos os elementos)
  - ✓ Empty (verifica se está vazia)

    pos retorna SE a pilha está vazia

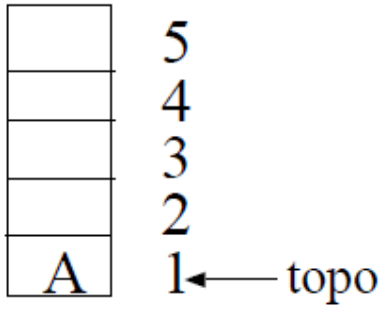
- **Condições pre e pos:** estando **pre** verificada, **pos** é verificada após a operação.

## Tópicos

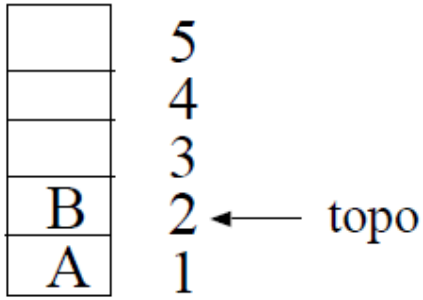
### Pilha (Stack)



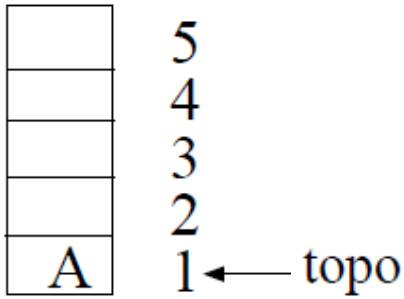

Pilha vazia
Pre == vazia
Pos == Pilha[1]




Push A
Pre != cheia
Pos: Pilha[1]==A



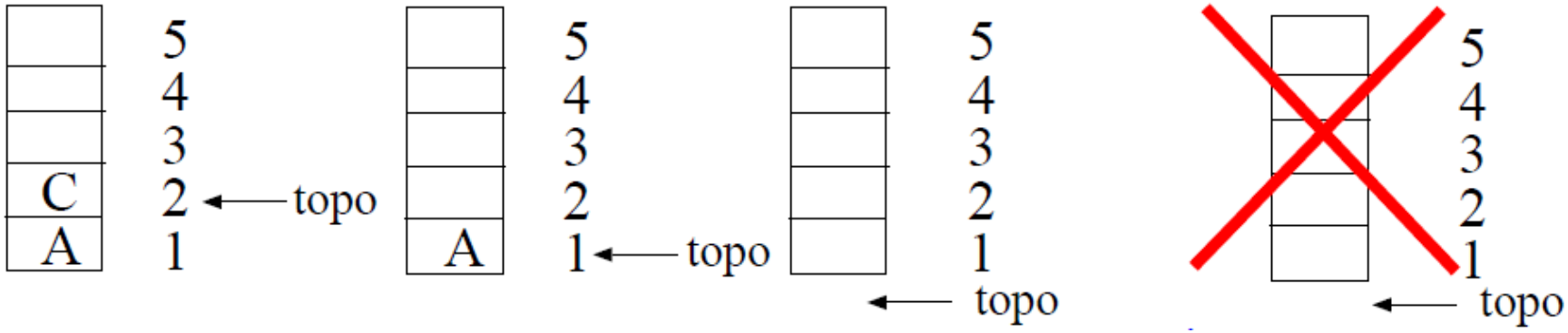

Push B
Pre != cheia
Pos: Pilha[2]==B




Pop B
Pre != vazia
Pos: Pilha[2]==[]

## Tópicos

### Pilha (Stack)




Push C
Pre != cheia
Pos:Pilha[2] ==C

Pop C
Pre != vazia
Pos: Pilha[2]==[]

Pop A
Pre != vazia
Pos: Pilha[1]==[]

Pop *ALGO*
Pre == vazia, logo underflow

## Tópicos

### Pilha (Stack) - Push

- **Inserção (push) na pilha P**

```
topo = topo + 1
se topo < M então              # M igual ao tamanho da pilha
        P[topo] = valor_a_ser_inserido
senão overflow
```

- Semelhante ao python

```
def push(stack, item):
        stack.append(item)
```

## Tópicos

### Pilha (Stack) - Pop

- **Remoção (pop) na pilha P**

```
se topo != 0 então
        valor_a_ser_removido = P[topo]     # a informação a ser
        topo = topo – 1                      removida é alocada
senão underflow                              na variável
```

- Semelhante ao python

```
def pop(stack):
        if not is_empty(stack): return stack.pop()
        else: return None              # Tratamento de pilha vazia
```

## Tópicos

### Pilha (Stack) - Pop

- **Exercício no caderno AGORA!!!**

Crie uma pilha vazia, em seguida faça as operações abaixo.
Determine a sequência correspondente as inserções (I) e remoções (R).

a) I(a), I(b), I(c), R, I(d), I(e), R, R, I(f), R, R, R, I(g)
b) I(a), I(b), I(c), R, R, I(d), R, R, I(e), R, I(f), R, R, I(g)

## Tópicos

### Pilha (Stack) - Pop

- **Exercício !!!**

Desenvolva um código em C para manipular uma pilha. Seu código deve inserir, remover, verificar se a pilha está cheia e verificar se a pilha está vazia. É necessário uma opção de entrada no main() para definir o tamanho máximo que a pilha (por exemplo 10 elementos) e os elementos que a pilha terá (n-1, ou seja, pelo exemplo 10-1 = 9 elementos). Em seguida, deve-se informar no terminal todos os elementos da pilha, se a pilha está vazia, se a pilha está cheia, incluir um elemento (qualquer valor) e informar no terminal a nova sequência, remover um elemento e informar no terminal a nova sequência. **PROIBIDO O USO DE BIBLIOTECAS COMO STACK**

***VALE NOTA!!!***